

Backend Fundamentals

1. Overview

Backend fundamentals are the concepts used to build the server-side part of an application. The backend receives requests from clients, processes business logic, talks to databases or other services, and sends responses back.

Definition:

A backend is the server-side system that exposes APIs, handles data, performs authentication and authorization, validates input, manages errors, logs events, and coordinates with databases or other services.

Why it matters:

- Most real applications need a backend to store data, enforce rules, and protect sensitive operations.
- Backend design affects scalability, reliability, security, performance, and maintainability.
- SDE interviews often test whether you understand how applications actually work beyond writing isolated algorithms.

Where it is used in real systems:

- E-commerce apps: product APIs, cart APIs, payment APIs, order APIs.
- Social media apps: feeds, posts, comments, likes, notifications.
- Banking systems: login, transaction validation, audit logs, fraud checks.
- Placement portals: user registration, resume upload, job applications, admin dashboards.
- SaaS products: user management, subscriptions, role-based access, analytics.

Why interviewers ask about it:

- To check if you can design clean APIs.
- To test security awareness around authentication and authorization.
- To see if you understand request-response flow.
- To evaluate practical engineering thinking: validation, errors, logging, pagination, and edge cases.

2. Core Idea

The core idea of backend development is simple:

A client asks for something, the backend checks whether the request is valid and allowed, performs the required work, and returns a structured response.

Intuition

Think of a backend server like a restaurant kitchen.

Restaurant	Backend System
Customer	Client/browser/mobile app
Waiter	API endpoint

Restaurant	Backend System
Menu	API contract
Kitchen	Business logic
Ingredients storage	Database
Bill/payment verification	Authentication and authorization
Complaint handling	Error handling
CCTV/register	Logging

The customer does not directly enter the kitchen. Similarly, a frontend should not directly access the database. It sends a request to the backend, and the backend controls what happens.

Small Example

Suppose a frontend wants to fetch a user's profile.

```
GET /api/users/42
Authorization: Bearer <jwt_token>
```

The backend may respond:

```
{
  "id": 42,
  "name": "Asha",
  "email": "asha@example.com"
}
```

Step-by-Step Explanation

1. Client sends an HTTP request.
2. Middleware checks request metadata such as authentication token, content type, rate limit, and logs.
3. Router maps the request to the correct controller or handler.
4. Validation checks input parameters, request body, and query parameters.
5. Business logic decides what should happen.
6. Backend reads from or writes to the database.
7. Response is returned in a standard format, usually JSON.
8. Logs and metrics record what happened.
9. Errors are converted into meaningful HTTP status codes and messages.

Text flow:

```
Client
|
```

```
  v
HTTP Request
  |
  v
Middleware -> Auth -> Validation -> Controller -> Service -> Database
  |
  v
HTTP Response in JSON
```

3. Important Subtopics

3.1 REST APIs

What it means:

REST stands for Representational State Transfer. A REST API exposes resources through URLs and uses HTTP methods to perform operations on those resources.

Example resources:

- `/users`
- `/users/42`
- `/orders`
- `/orders/1001/items`

Common HTTP methods:

Method	Meaning	Example
GET	Read data	GET <code>/users/42</code>
POST	Create data	POST <code>/users</code>
PUT	Replace data	PUT <code>/users/42</code>
PATCH	Partially update data	PATCH <code>/users/42</code>
DELETE	Delete data	DELETE <code>/users/42</code>

Why it matters:

REST gives a predictable structure to backend APIs. It helps frontend, mobile, and backend teams communicate using a shared contract.

Example:

```
POST /api/products
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 1200
}
```

Common interview angle:

Interviewers may ask you to design APIs for a system like URL shortener, library management, food delivery, or social media posts.

Expected points:

- Use nouns in URLs, not verbs.
- Use correct HTTP methods.
- Use status codes properly.
- Keep APIs stateless.
- Version APIs when needed, for example `/api/v1/users`.

3.2 JSON

What it means:

JSON, or JavaScript Object Notation, is a lightweight data format used to send structured data between client and server.

Why it matters:

Most REST APIs use JSON because it is simple, readable, language-independent, and easy to parse.

Example:

```
{
  "id": 101,
  "title": "Backend Engineer",
  "skills": ["Node.js", "SQL", "REST"],
  "isRemote": true
}
```

Common interview angle:

Interviewers may ask how JSON differs from objects, XML, or plain text.

Important points:

- JSON is a text format, not a programming language object.
- JSON supports strings, numbers, booleans, arrays, objects, and `null`.
- JSON does not support comments, functions, or undefined values.

3.3 Authentication

What it means:

Authentication verifies who the user is.

Example: username and password login.

Why it matters:

Without authentication, the backend cannot know which user is making the request.

Example:

```
User enters email + password
Backend verifies credentials
Backend returns session cookie or token
Client uses that credential in future requests
```

Common interview angle:

Interviewers often ask:

- Authentication vs authorization.
- Session-based auth vs token-based auth.
- How login works internally.
- How passwords should be stored.

Must-remember:

- Store password hashes, not plain passwords.
- Use strong hashing algorithms such as bcrypt, Argon2, or scrypt.
- Use HTTPS to protect credentials in transit.

3.4 Authorization

What it means:

Authorization checks what an authenticated user is allowed to do.

Why it matters:

Even if a user is logged in, they should not access everything.

Example:

```
A normal user can view their own orders.
An admin can view all orders.
```

Common interview angle:

Interviewers may ask how you prevent a user from accessing another user's data.

Expected answer:

- Check user identity from a trusted auth source.

- Check ownership or role before returning data.
- Never trust user-provided IDs alone.

3.5 JWT

What it means:

JWT stands for JSON Web Token. It is a compact token format used to represent claims about a user.

A JWT has three parts:

```
header.payload.signature
```

Part	Purpose
Header	Token type and signing algorithm
Payload	Claims such as user ID, role, expiry
Signature	Verifies token integrity

Why it matters:

JWT is commonly used in stateless authentication for APIs.

Example payload:

```
{
  "sub": "42",
  "role": "user",
  "exp": 1760000000
}
```

Common interview angle:

Interviewers ask whether JWT is encrypted.

Correct answer:

- A normal JWT is signed, not encrypted.
- Anyone with the token can decode the header and payload.
- Sensitive data should not be stored inside a JWT.
- The signature proves the token was not modified.

3.6 OAuth Basics

What it means:

OAuth is an authorization framework that allows one application to access resources from another service with user consent, without sharing the user's password.

Why it matters:

OAuth is used in "Login with Google", "Login with GitHub", and third-party integrations.

Real-world analogy:

You give a valet key to park your car. The valet can drive the car for a limited purpose but cannot open your house or access your bank account.

Example:

```
User clicks Login with Google
App redirects user to Google
User approves
Google sends authorization code to app
App exchanges code for tokens
App uses token to identify user or access allowed data
```

Common interview angle:

Interviewers may ask the difference between OAuth and JWT.

Short answer:

- OAuth is a protocol/framework for delegated authorization.
- JWT is a token format.
- OAuth systems may use JWTs, but OAuth and JWT are not the same thing.

3.7 Middleware

What it means:

Middleware is code that runs between receiving a request and sending a response.

Why it matters:

Middleware helps separate cross-cutting concerns such as logging, authentication, validation, rate limiting, and error handling.

Example:

```
Request -> Logging Middleware -> Auth Middleware -> Route Handler -> Response
```

Common interview angle:

Interviewers may ask where you would place authentication, request logging, or error handling.

Expected answer:

- Authentication should usually run before protected route handlers.
- Logging can run early to capture all requests.
- Error middleware runs after handlers to standardize failures.

3.8 Validation

What it means:

Validation checks whether incoming data is correct, complete, and safe before processing it.

Why it matters:

Invalid input can cause bugs, security issues, corrupted data, or application crashes.

Example:

```
{
  "email": "student@example.com",
  "age": 21
}
```

Validation rules:

- `email` must be a valid email.
- `age` must be a positive integer.
- Required fields must be present.
- Unknown or dangerous fields may be rejected.

Common interview angle:

Interviewers may ask why frontend validation is not enough.

Correct answer:

- Frontend validation improves user experience.
- Backend validation is mandatory for security and correctness.
- Attackers can bypass the frontend and directly call APIs.

3.9 Pagination

What it means:

Pagination splits a large result set into smaller chunks.

Why it matters:

Returning thousands or millions of rows in one API response is slow, memory-heavy, and bad for user experience.

Example:

```
GET /api/products?page=2&limit=20
```

Response:

```
{
  "data": [
    { "id": 21, "name": "Mouse" },
  ]
}
```

```
{ "id": 22, "name": "Monitor" }  
],  
"page": 2,  
"limit": 20,  
"total": 153  
}
```

Common interview angle:

Interviewers may ask offset pagination vs cursor pagination.

Key point:

- Offset pagination is simple but can become slow for deep pages.
- Cursor pagination is better for large, frequently changing datasets.

3.10 Logging

What it means:

Logging records important events that happen in a backend system.

Why it matters:

Logs help debug issues, monitor production behavior, investigate security incidents, and understand user flows.

Example log fields:

```
{  
  "timestamp": "2026-07-09T10:00:00Z",  
  "level": "info",  
  "method": "GET",  
  "path": "/api/orders",  
  "status": 200,  
  "durationMs": 42,  
  "requestId": "req_123"  
}
```

Common interview angle:

Interviewers may ask what not to log.

Do not log:

- Passwords
- Access tokens
- Refresh tokens
- Credit card numbers
- Sensitive personal information unless properly masked

3.11 Error Handling

What it means:
Error handling means detecting failures and returning consistent, meaningful responses.

Why it matters:
Good error handling improves debugging, user experience, and system reliability.

Example:

```
{
  "error": {
    "code": "USER_NOT_FOUND",
    "message": "User not found"
  }
}
```

Common interview angle:
Interviewers may ask how to map errors to HTTP status codes.

Status Code	Meaning	Example
400	Bad request	Invalid input
401	Unauthorized	Missing or invalid login token
403	Forbidden	Logged in but not allowed
404	Not found	User does not exist
409	Conflict	Duplicate email
429	Too many requests	Rate limit exceeded
500	Internal server error	Unexpected backend failure

4. Real-World Example

Consider a backend API for a placement portal where students apply for jobs.

Scenario: Student Applies for a Job

```
POST /api/jobs/501/applications
Authorization: Bearer <jwt_token>
Content-Type: application/json

{
  "resumeId": "resume_789",
  "coverLetter": "I am interested in this role."
}
```

Backend flow:

- 1. Request reaches the server.
- 2. Logging middleware creates a request ID.
- 3. Auth middleware verifies the JWT.
- 4. Authorization checks whether the user has the `student` role.
- 5. Validation checks whether `resumeId` exists and `coverLetter` length is acceptable.
- 6. Service checks whether the job is open.
- 7. Service checks whether the student already applied.
- 8. Database stores the application.
- 9. Response returns `201 Created`.

Example response:

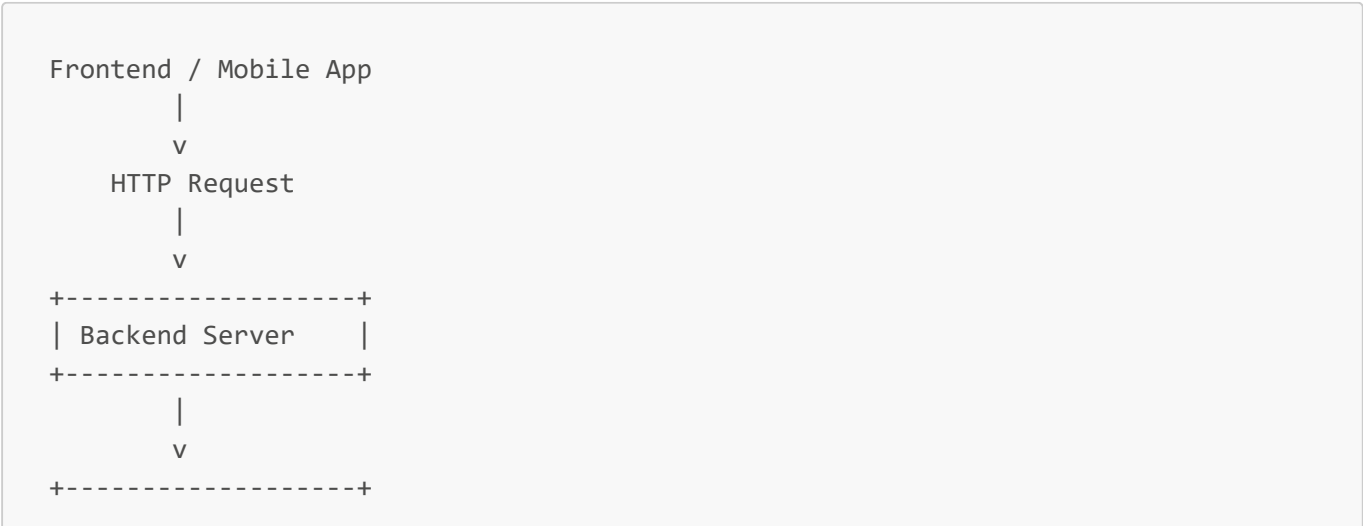
```
{
  "applicationId": "app_10001",
  "jobId": 501,
  "status": "submitted"
}
```

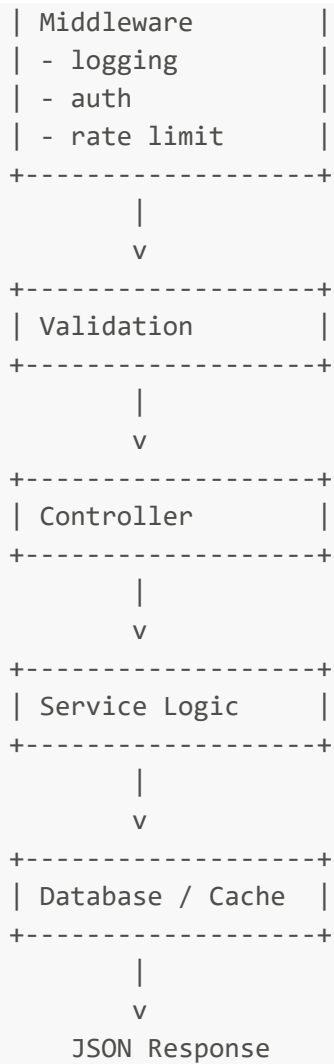
Possible errors:

Case	Status
Missing token	401 Unauthorized
User is recruiter, not student	403 Forbidden
Invalid resume ID	400 Bad Request
Job not found	404 Not Found
Already applied	409 Conflict
Server crash	500 Internal Server Error

5. Diagrams / Mental Models

Backend Request Lifecycle





Auth vs Authorization Mental Model

Authentication: "Who are you?"

Authorization: "What are you allowed to do?"

Example:

Login successful -> authentication passed

Trying to delete another user's account -> authorization failed

JWT Structure

```
xxxxx.yyyyy.zzzzz
|       |       |
|       |       +--- Signature
|       +----- Payload / claims
+----- Header
```

REST Resource Design

Bad URL	Better REST URL
<code>/getUser?id=10</code>	<code>GET /users/10</code>
<code>/createProduct</code>	<code>POST /products</code>
<code>/deleteOrder?id=5</code>	<code>DELETE /orders/5</code>
<code>/updateProfile</code>	<code>PATCH /users/me</code>

6. Common Interview Questions

1. What is a REST API?

Answer:

A REST API is an HTTP-based interface where resources are represented by URLs and operations are performed using HTTP methods such as `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`.

Key points interviewer expects:

- Resource-based URLs.
- Correct use of HTTP methods.
- Stateless communication.
- JSON responses are common.
- HTTP status codes matter.

Common mistakes:

- Saying REST means only JSON.
- Using verbs in every URL, such as `/getUser`.
- Ignoring status codes.

2. What is the difference between authentication and authorization?

Answer:

Authentication verifies identity. Authorization verifies permissions.

Example:

Authentication: User logs in successfully.
Authorization: User is allowed to access only their own profile.

Key points interviewer expects:

- AuthN means identity.
- AuthZ means access control.
- Authorization usually happens after authentication.

Common mistakes:

- Treating both as the same thing.
- Assuming a logged-in user can access everything.

3. What is JWT and how does it work?

Answer:

JWT is a signed token format containing claims. The server creates a token after login. The client sends it in future requests. The server verifies the signature and checks claims such as expiry and user ID.

Key points interviewer expects:

- JWT has header, payload, and signature.
- Payload is encoded, not necessarily encrypted.
- Signature prevents tampering.
- Expiry should be checked.

Common mistakes:

- Saying JWT is always encrypted.
- Storing passwords or secrets in JWT payload.
- Ignoring token expiry.

4. What is middleware?

Answer:

Middleware is code that runs during the request-response cycle before or after route handlers.

Key points interviewer expects:

- Used for logging, auth, validation, rate limiting, parsing, and error handling.
- Helps avoid repeating common logic in every route.

Common mistakes:

- Thinking middleware is only for authentication.
- Putting business logic entirely inside middleware.

5. Why is backend validation necessary if frontend validation exists?

Answer:

Frontend validation improves user experience, but it can be bypassed. Backend validation is required because clients are not trusted.

Key points interviewer expects:

- Attackers can call APIs directly.
- Backend protects database correctness.
- Validation prevents crashes and security issues.

Common mistakes:

- Trusting browser-side checks.
- Validating only required fields and ignoring type, length, and range.

6. What is pagination and why is it needed?

Answer:

Pagination splits large data into smaller pages. It improves performance, reduces response size, and makes APIs easier to consume.

Key points interviewer expects:

- Prevent loading huge data at once.
- Use `limit`, `offset`, `page`, or cursor.
- Include metadata when useful.

Common mistakes:

- Returning all database rows.
- Not defining stable sorting.
- Ignoring maximum page size limits.

7. Difference between `PUT` and `PATCH`?

Answer:

`PUT` usually replaces the entire resource. `PATCH` updates part of a resource.

Example:

```
PUT /users/42
```

May replace the whole user object.

```
PATCH /users/42
```

May update only the user's name.

Key points interviewer expects:

- **PUT** is full replacement.
- **PATCH** is partial update.
- Both should be idempotent when designed properly, especially **PUT**.

Common mistakes:

- Using **POST** for every update.
- Saying **PUT** and **PATCH** are identical.

8. What are common HTTP status codes used in backend APIs?**Answer:**

Common status codes include:

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
429	Too Many Requests
500	Internal Server Error

Key points interviewer expects:

- Use accurate status codes.
- Distinguish **401** and **403**.
- Do not return **200** for errors.

Common mistakes:

- Returning **500** for validation errors.
- Returning detailed internal stack traces to clients.

9. What is OAuth?**Answer:**

OAuth is a delegated authorization framework. It lets an app access a user's resources from another service without asking for the user's password.

Key points interviewer expects:

- Used for third-party login and integrations.
- Uses user consent.
- Often involves authorization code and tokens.
- OAuth is not the same as JWT.

Common mistakes:

- Saying OAuth is just login.
- Confusing OAuth with password sharing.

10. What should you log in a backend service?**Answer:**

Log useful operational data such as request ID, timestamp, route, method, status code, latency, error code, and user ID when safe.

Key points interviewer expects:

- Structured logs are better than random text logs.
- Logs help debugging and monitoring.
- Sensitive data must be masked.

Common mistakes:

- Logging passwords or tokens.
- Logging too little to debug production issues.
- Logging too much and increasing cost or privacy risk.

11. How should an API return errors?**Answer:**

An API should return consistent error responses with proper status codes and useful error messages.

Example:

```
{
  "error": {
    "code": "INVALID_EMAIL",
    "message": "Email format is invalid"
  }
}
```

Key points interviewer expects:

- Consistent structure.
- Correct HTTP status code.
- Do not expose internal stack traces.
- Include request ID for debugging when possible.

Common mistakes:

- Returning plain strings randomly.
- Exposing database errors directly.

12. What does stateless mean in REST?

Answer:

Stateless means each request contains all information needed to process it. The server does not rely on hidden client-specific state from previous requests.

Key points interviewer expects:

- Each request is independent.
- Helps scalability because any server instance can handle the request.
- Tokens are often sent with each request.

Common mistakes:

- Thinking stateless means no database.
- Thinking sessions can never exist in web apps.

7. Deep-Dive Questions

1. How would you design authentication for a production REST API?

Use HTTPS, hash passwords with bcrypt/Argon2/scrypt, issue short-lived access tokens, use refresh tokens carefully, validate token expiry, rotate secrets when needed, and protect sensitive routes with authorization checks.

A strong answer mentions:

- Password hashing, not encryption.
- Access token expiry.
- Refresh token storage and revocation.
- Rate limiting login attempts.
- Secure cookies or secure storage depending on client type.

2. How do you revoke JWTs if JWT is stateless?

JWTs are hard to revoke if only stateless verification is used. Common solutions include:

- Short access token expiry.
- Refresh token revocation in database.
- Token blacklist until expiry.
- Token version field stored in database.
- Re-issue tokens after password change or logout.

Trade-off:

- Pure stateless JWT is simple and scalable.
- Revocation needs some server-side state.

3. How would you prevent users from accessing other users' data?

Do not trust IDs from the client. After authentication, derive the user ID from the trusted token/session. Then check ownership or permissions before accessing data.

Example:

```
Bad: GET /users/42/orders and trust that the caller owns user 42
Good: GET /me/orders and derive user ID from authenticated identity
```

For admin routes, use role-based or permission-based authorization.

4. When should you use cursor pagination instead of offset pagination?

Use cursor pagination when:

- Data is large.
- Records are frequently inserted or deleted.
- Users scroll through feeds or timelines.
- Deep pagination must be efficient.

Offset pagination can skip or duplicate records if new rows are inserted while the user is paging. Cursor pagination uses a stable marker such as `createdAt` plus `id`.

5. How do you design error handling in a backend project?

Use typed or categorized errors in service code, map them to HTTP status codes in a central error handler, log internal details safely, and return client-safe messages.

Good design:

```
ValidationError -> 400
AuthenticationError -> 401
AuthorizationError -> 403
NotFoundError -> 404
ConflictError -> 409
UnknownError -> 500
```

Avoid scattering random `try/catch` blocks everywhere without a consistent response format.

8. Comparison Tables

REST vs GraphQL

Point	REST	GraphQL
API shape	Multiple resource endpoints	Single endpoint commonly used

Point	REST	GraphQL
Data fetching	Server decides response shape	Client asks for specific fields
Simplicity	Simple and widely understood	More flexible but more complex
Over-fetching	Possible	Reduced
Under-fetching	Possible	Reduced
Caching	Easier with HTTP caching	More custom caching needed
Interview note	Know REST first	Useful for advanced discussions

Authentication vs Authorization

Point	Authentication	Authorization
Question answered	Who are you?	What can you access?
Happens when	Login or token verification	Before protected operation
Example	Password verified	Role <code>admin</code> required
Failure code	Usually <code>401</code>	Usually <code>403</code>

JWT vs Session-Based Auth

Point	JWT Auth	Session Auth
State	Often stateless on server	Server stores session
Client stores	Token	Session ID cookie
Revocation	Harder unless tracked	Easier by deleting session
Payload	Can contain claims	Session data stored server-side
Scaling	Easy if purely stateless	Needs shared session store
Risk	Token leakage is dangerous	Cookie/session hijacking is dangerous

`401 Unauthorized` vs `403 Forbidden`

Status	Meaning	Example
<code>401</code>	Not authenticated	Missing or invalid token
<code>403</code>	Authenticated but not allowed	Normal user tries admin route

PUT vs PATCH

Point	PUT	PATCH
Purpose	Replace full resource	Update part of resource

Point	PUT	PATCH
Request body	Usually complete object	Partial object
Idempotency	Expected to be idempotent	Can be idempotent depending on design
Example	Replace full profile	Change only phone number

Offset Pagination vs Cursor Pagination

Point	Offset Pagination	Cursor Pagination
Example	<code>?page=3&limit=20</code>	<code>?after=cursor123&limit=20</code>
Simplicity	Very simple	More complex
Deep pages	Can be slow	Efficient
Changing data	Can cause duplicates/skips	More stable
Best for	Admin tables, small lists	Feeds, timelines, large data

Validation vs Sanitization

Point	Validation	Sanitization
Meaning	Checks if input is acceptable	Cleans or transforms input
Example	Age must be positive	Trim whitespace from email
Goal	Reject bad data	Normalize data
Interview trap	Validation does not automatically make input safe everywhere	Sanitization does not replace validation

Logging vs Monitoring

Point	Logging	Monitoring
Meaning	Records events	Tracks system health
Data	Request logs, error logs	Metrics, alerts, dashboards
Example	<code>POST /login failed</code>	Error rate above 5%
Use	Debugging details	Operational visibility

9. Common Mistakes

- Using `POST` for every API operation.
- Designing URLs with verbs instead of resources.
- Returning `200 OK` even when an error occurred.
- Confusing authentication with authorization.
- Assuming JWT payload is secret.

- Storing passwords in plain text.
- Trusting frontend validation.
- Not validating query parameters and path parameters.
- Returning huge lists without pagination.
- Logging sensitive values such as passwords, tokens, or card details.
- Exposing stack traces and internal database errors to users.
- Not checking ownership of resources.
- Using only role checks when object-level permission checks are also needed.
- Ignoring token expiry.
- Not setting maximum limits for pagination.
- Forgetting consistent error response format.

10. Edge Cases / Special Cases

REST API Edge Cases

- **DELETE** can return **204 No Content** if deletion succeeds and no body is needed.
- **POST** is not generally idempotent. Repeating it may create duplicate resources.
- **PUT** should usually be idempotent. Repeating the same request should produce the same final state.
- APIs should handle duplicate submissions, especially payments and applications.

JSON Edge Cases

- JSON does not support comments.
- **undefined** is not valid JSON.
- Dates are usually represented as strings, commonly ISO 8601.
- Large integers may lose precision in some JavaScript clients.

JWT Edge Cases

- JWT expiry must be checked.
- JWT signature must be verified.
- Do not accept **alg: none**.
- Use proper secret or public/private key management.
- Token leakage is serious because bearer tokens can be used by whoever holds them.

OAuth Edge Cases

- Redirect URI must be validated.
- Authorization code should be short-lived.
- State parameter helps prevent CSRF attacks.
- Scopes should be minimal.

Pagination Edge Cases

- Always enforce maximum **limit**.
- Use stable sorting, such as **createdAt** plus **id**.
- New records inserted during pagination can cause duplicate or missing results with offset pagination.

Error Handling Edge Cases

- Do not expose internal exception messages.
- Some errors should be logged at `warn`, not `error`.
- Client errors like `400` are often not backend bugs.
- A `404` can be used to avoid revealing whether a protected resource exists.

11. How to Explain in Interview

Backend APIs are the server-side interface through which clients access application data and operations. In a typical REST backend, resources are exposed through URLs, HTTP methods define actions, JSON is used for request and response bodies, middleware handles common concerns like logging and authentication, validation protects the system from bad input, and errors are returned using proper HTTP status codes. For security, authentication verifies who the user is, authorization checks what they can do, and tokens like JWTs or OAuth flows are used depending on the use case.

12. Quick Revision Notes

Key Definitions

Term	Meaning
Backend	Server-side system handling APIs, business logic, and data access
REST	Resource-based API style using HTTP methods
JSON	Text-based structured data format
Authentication	Verifies user identity
Authorization	Verifies user permissions
JWT	Signed token format containing claims
OAuth	Delegated authorization framework
Middleware	Code running during request-response lifecycle
Validation	Checking input correctness and safety
Pagination	Splitting large result sets
Logging	Recording system events
Error handling	Returning controlled and useful failures

Important Points

- REST uses resources, methods, and status codes.
- JSON is a data format, not a database or language object.
- Authentication is not the same as authorization.
- JWT is signed, not automatically encrypted.
- OAuth is a framework, JWT is a token format.
- Backend validation is mandatory.

- Pagination avoids large responses.
- Logs should help debugging but must not leak secrets.
- Error responses should be consistent.

Common Comparisons

Comparison	Main Difference
AuthN vs AuthZ	Identity vs permission
JWT vs Session	Stateless token claims vs server-side session
OAuth vs JWT	Authorization framework vs token format
PUT vs PATCH	Full replacement vs partial update
401 vs 403	Not logged in vs not allowed
Offset vs Cursor	Simple page number vs stable position marker

Must-Remember Facts

- Never store plain passwords.
- Never trust client input.
- Never put secrets in JWT payload.
- Use HTTPS for authentication.
- Use correct status codes.
- Enforce pagination limits.
- Log request IDs for debugging.
- Return client-safe error messages.

Interview Traps

- Saying JWT is encrypted.
- Saying OAuth and JWT are the same.
- Saying frontend validation is enough.
- Returning 500 for all errors.
- Forgetting authorization after authentication.
- Ignoring object ownership checks.

13. Practice Tasks

Task 1: Design REST APIs for a Student Placement Portal

Design endpoints for:

- Student registration.
- Student login.
- Viewing jobs.
- Applying for a job.
- Viewing application status.

- Recruiter posting a job.

Expected examples:

```
POST /api/students
POST /api/auth/login
GET /api/jobs?page=1&limit=20
POST /api/jobs/{jobId}/applications
GET /api/me/applications
POST /api/recruiter/jobs
```

Task 2: Implement Simple Express Middleware

Write middleware that logs method, URL, status code, and response time.

Pseudo-code:

```
function requestLogger(req, res, next) {
  const start = Date.now();

  res.on("finish", () => {
    console.log({
      method: req.method,
      url: req.url,
      status: res.statusCode,
      durationMs: Date.now() - start
    });
  });

  next();
}
```

Task 3: Validate a Signup Request

Given this request:

```
{
  "email": "abc",
  "password": "123",
  "age": -5
}
```

Identify validation errors:

- Email format invalid.
- Password too short.
- Age must be positive.

Task 4: Design Error Responses

Create standard error responses for:

- Invalid email.
- User not found.
- Unauthorized request.
- Duplicate application.
- Internal server error.

Example:

```
{
  "error": {
    "code": "DUPLICATE_APPLICATION",
    "message": "You have already applied for this job"
  }
}
```

Task 5: Compare Pagination Strategies

For a social media feed, decide whether offset or cursor pagination is better.

Expected answer:

- Cursor pagination is better.
- Feeds change frequently.
- New posts can cause offset pagination to skip or duplicate items.

Task 6: Trace a Login Flow

Explain every step from entering email/password to calling a protected API.

Expected steps:

```
User submits credentials
Backend validates input
Backend verifies password hash
Backend issues token/session
Client stores credential safely
Client sends credential in future requests
Backend verifies credential
Backend checks authorization
Backend returns protected data
```

Task 7: Identify Status Codes

Choose correct status codes:

Situation	Status
New user created	201
Invalid request body	400
Missing token	401
User lacks admin role	403
Product not found	404
Duplicate email	409
Too many login attempts	429

Task 8: Secure a JWT-Based API

List improvements:

- Short-lived access tokens.
- Refresh token rotation.
- HTTPS only.
- Check expiry.
- Avoid sensitive claims.
- Use strong signing keys.
- Add logout/revocation strategy.

14. Final Cheat Sheet

Core Definition

Backend fundamentals cover how servers expose APIs, receive JSON requests, authenticate users, authorize actions, validate input, paginate data, log behavior, and return errors safely.

Why It Matters

Backend quality decides whether an application is secure, scalable, debuggable, and reliable.

Most Asked Questions

Question	Short Answer
What is REST?	Resource-based API style using HTTP methods
What is JSON?	Text format for structured data
Auth vs authorization?	Identity vs permission
What is JWT?	Signed token containing claims
Is JWT encrypted?	Not by default; usually only signed
What is OAuth?	Delegated authorization framework

Question	Short Answer
What is middleware?	Code in the request-response pipeline
Why validation?	Never trust client input
Why pagination?	Avoid huge slow responses
401 vs 403?	Not authenticated vs not allowed

Common Comparisons

A	B	Difference
GET	POST	Read vs create/submit
PUT	PATCH	Full replace vs partial update
JWT	Session	Token claims vs server-side session
OAuth	JWT	Framework vs token format
Offset pagination	Cursor pagination	Page number vs stable marker
Validation	Sanitization	Check vs clean

One-Line Interview Answer

A backend REST API exposes server-side resources over HTTP, usually exchanges JSON, uses middleware for cross-cutting concerns, authenticates and authorizes users, validates all input, paginates large data, logs important events, and returns consistent errors with proper status codes.